

Lecture 3

Reading and running the FLINT certificate

Detailed explanations accompanying the torsion manuscript

13 September 2026

Contents

1	What the executable has to prove	2
2	The finite element data entering one entry check	2
3	Arb arithmetic: enclosures rather than rounded answers	3
4	The entry verifier, function by function	4
4.1	Reading a finite conforming mesh	4
4.2	Proving the exact regular fan and the directions	4
4.3	Exact element integration and coefficient bounds	5
4.4	Residuals, reference values, and the main calculation	6
5	The Fourier verifier, function by function	7
6	How the rotation-based driver obtains the candidates	8
6.1	The FreeFEM problems	8
6.2	rotation_cache.py	8
6.3	certificate_build.py	9
6.4	certify_with_rotations.py	9
7	Other validation utilities and their precise roles	10
7.1	residual_cert.c	10
7.2	second_variation_cert.c	10
7.3	majorant_cert.c	10
7.4	Drivers, summaries, and checks	10
8	Running a certificate yourself	11
9	What the archived computations establish	11
9.1	Time and the meaning of the three-solve optimization	12
9.2	Where the evidence is stored	13

1 What the executable has to prove

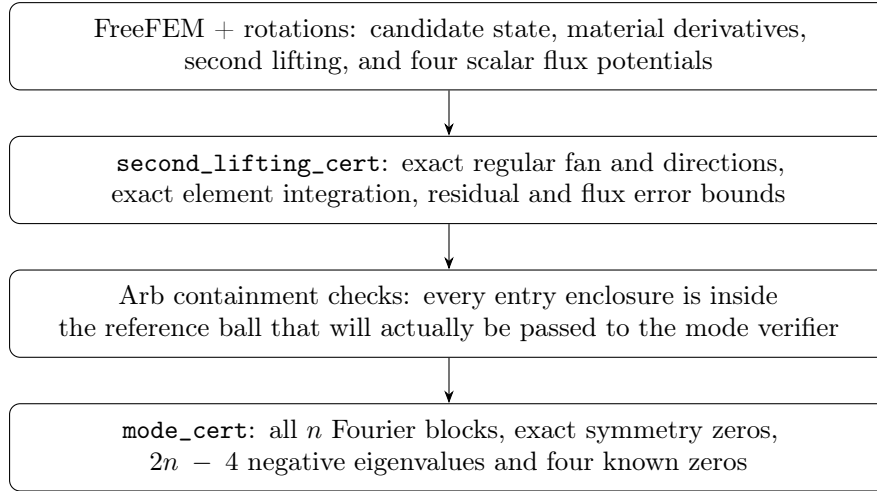
Aim. Read the maintained implementation as a sequence of mathematical assertions. A suggested 90-minute session devotes 15 minutes to the proof contract, 35 to the entry verifier, 20 to the Fourier verifier, and 20 to reproduction and results. The function descriptions also serve as a reference for a longer practical session with the source open.

For the circumradius-one regular polygon, let

$$-\Delta u = 1, \quad u|_{\partial P_n} = 0, \quad J = \frac{1}{2} \int_{P_n} u, \quad \mathcal{F} = J/|P_n|^2.$$

The conclusion sought is that the vertex Hessian of $\mathcal{F}$ has exactly $2n - 4$ strictly negative eigenvalues and the four zero eigenvalues imposed by two translations, rotation, and dilation. Lecture 1 explains why this implies strict local maximality modulo similarities. Lecture 2 supplies the error bounds for the individual Hessian entries.

There are two different finite computations. The first encloses exact PDE-dependent Hessian entries. The second converts those enclosures into eigenvalue signs. The following diagram is also the order in which to read the implementation.



FreeFEM and NumPy propose finite element functions. Their linear solves, floating geometry, and FFTs need not be exact: the C verifier checks the geometry and directions, then bounds the errors of the supplied functions. The mathematical identities, the implementation of the two verifiers, and FLINT/Arb's arithmetic remain part of the trusted argument. This is a computer-assisted proof, rather than a proof object checked by a formal proof assistant.

The current source root is `MaxTorsionValidation/`. The accompanying `source_index.md` gives function locations, and `sources.json` records the hashes of the files used for these notes. Descriptions below refer to that snapshot, so a reader can distinguish it from sources saved inside an older computation archive.

2 The finite element data entering one entry check

Write $\vartheta = 2\pi/n$. Subdivide each coarse fan triangle

$$T_j = \text{conv}\{0, (\cos j\vartheta, \sin j\vartheta), (\cos((j+1)\vartheta), \sin((j+1)\vartheta))\}$$

uniformly with m segments on each side. Then

$$\#\text{triangles} = nm^2, \quad \#\text{nodes} = 1 + \frac{nm(m+1)}{2}, \quad \#\text{boundary nodes} = nm.$$

Every fine triangle is contained in one coarse sector. Thus the gradients of the fan extensions and the pullback coefficients are constant on each fine triangle. This fitted structure is used both in the regularity-based rate discussion and in the exact coefficient reconstruction.

An export starts with

```
TORSION_SECOND_V2 n m k qa qp ra rp nv nt
```

Here k is the Fourier index; **qa**, **ra** select radial (0) or tangential (1) directions, and **qp**, **rp** select cosine (0) or sine (1). The final two integers are the node and triangle counts. Next come the node coordinates, the three node indices of each triangle, and twelve nodal arrays, in this order:

```
qx qy rx ry u uq ur zh psi0 psiq psir psiz
```

The first four arrays are components of the fan extensions θ_q, θ_r . The next four specify conforming candidates for the state, two first material derivatives, and the second lifting. The final four specify continuous piecewise affine scalar potentials whose curls are used in the equilibrated fluxes. An array named **u** is therefore a computed candidate, not the exact torsion solution.

For a nonzero index other than the even Nyquist index, the normalized real directions are

$$\begin{aligned} r_{k,j}^c &= \sqrt{2/n} \cos(kj\vartheta) (\cos j\vartheta, \sin j\vartheta), & \text{radial, cosine,} \\ t_{k,j}^c &= \sqrt{2/n} \cos(kj\vartheta) (-\sin j\vartheta, \cos j\vartheta), & \text{tangential, cosine,} \\ t_{k,j}^s &= \sqrt{2/n} \sin(kj\vartheta) (-\sin j\vartheta, \cos j\vartheta), & \text{tangential, sine.} \end{aligned}$$

Using the Fourier convention of Lecture 1, the four requested entries are the following real Hessian pairings:

Entry name	q	r	Exact value of $D^2\mathcal{F}[q, r]$
rr	r_k^c	r_k^c	α_k
tt	t_k^c	t_k^c	β_k
rt_re	r_k^c	t_k^c	$\operatorname{Re}(B_k)_{12} = 0$
rt_im	r_k^c	t_k^s	$\operatorname{Im}(B_k)_{12} = \gamma_k$

For n even and $k = n/2$, replace $\sqrt{2/n}$ by $1/\sqrt{n}$ and omit the zero sine direction. Only the two diagonal pairings are then needed.

3 Arb arithmetic: enclosures rather than rounded answers

An Arb real ball represents a midpoint with a rigorous absolute error radius. Arithmetic propagates enclosures; increasing the precision reduces rounding error but does not remove finite element error. The maintained driver uses 192 bits in both verification stages. The relevant documented operations are: **arb_set_str** for decimal input, **arb_add_error** for enlarging a radius, **arb_contains** for containment, and **arb_is_negative** for strict negativity of the whole ball [1].

In particular, a negative midpoint is insufficient. If an enclosure is $[-3 \cdot 10^{-5}, 10^{-6}]$, its sign is unresolved. Similarly, printed digits are not the internal ball. The driver may use printed output to propose a convenient radius, but the entry verifier must subsequently prove that the exact reference ball contains its internal enclosure.

The code uses **arb_sqrtpos** only for expressions mathematically known to be nonnegative, such as sums of squares. Intersecting their enclosure with the nonnegative half-line is justified by that identity; it would not justify taking a square root of an arbitrary uncertain input. The input readers reject nonfinite numbers before they are used in the certificate.

4 The entry verifier, function by function

Open `MaxTorsionValidation/flint/second_lifting_cert.c`. The program does not solve a large interval linear system. It integrates the supplied piecewise affine fields and evaluates the error inequalities of Lecture 2. Its functions fall into four short groups, followed by the main calculation.

4.1 Reading a finite conforming mesh

die. Print an explanation and exit with status 2. A failed input assertion is an error, not a certificate with a large radius.

cmp_edge and cmp_tri_key. Compare sorted vertex pairs and triples lexicographically. Sorting makes edge multiplicities and duplicate triangles detectable independently of their order in the exported file.

dsu_root and dsu_join. Maintain connected components using a disjoint-set data structure, with path compression. They supply the connectivity part of the mesh check.

certify_mesh_topology. Check valid and distinct triangle vertices, reject duplicate triangles, require all nodes to be used and connected, count edges with one or two incident triangles, and check the boundary degrees, boundary count, and Euler relation. In strict mode the node and triangle counts must equal the regular-fan formulas above. The function returns the boundary mask. These combinatorial conditions are necessary; the later equality with the complete canonical triangulation establishes the particular mesh required by the proof.

read_ball and read_field. Read a finite decimal as an Arb ball and optionally enlarge it by a specified radius; repeat for a nodal array. Geometry and direction candidates are initially given radii 10^{-13} . PDE and potential coefficients are read with no additional modelling radius: the decimal coefficients define the candidate finite element function, with conversion rounding enclosed by Arb. Truncation and nonfinite data are rejected.

assemble_triangle_geometry. Form the signed determinant of the two triangle edge vectors, prove it does not contain zero, and compute the area and the gradients of the three barycentric basis functions. The signed determinant is used for gradients; its absolute value is used for area. Geometry is assembled again after the exact regular coordinates have replaced the candidate coordinates.

4.2 Proving the exact regular fan and the directions

canonical_gid. Give each lattice node of the regular fan a single integer index. The centre has one index; nodes on shared rays have the same index when described from either neighboring sector. This is how the program avoids counting a shared ray twice.

canonical_tri_key. Sort the three canonical node indices of a triangle. This removes an irrelevant orientation or input-order choice when comparing triangulations.

validate_regular_inputs. Reconstruct the exact node set

$$x = \frac{s}{m}(\cos j\vartheta, \sin j\vartheta) + \frac{t}{m}(\cos((j+1)\vartheta), \sin((j+1)\vartheta)), \quad s, t \geq 0, \quad s + t \leq m.$$

The integer pair (s, t) is a lattice address, not a new angle notation. Floating calculations nominate an address; Arb then proves that the candidate coordinate balls contain the trigonometric coordinates of that address. The assignment must be a bijection. The boundary mask must agree with the canonical boundary, and the sorted list of triangles must equal the full canonical list, not merely have the correct cardinality.

The function also reconstructs the normalized Fourier directions at each node and proves their containment in the exported direction balls. It finally replaces geometry and direction arrays by these analytic enclosures. Each triangle has exact area $\sin \vartheta / (2m^2)$ and the total area is $n \sin \vartheta / 2$. Consequently the 10^{-13} input radii serve to identify the intended exact data; they do not certify an arbitrary perturbation of the polygon or enter as a shape uncertainty.

4.3 Exact element integration and coefficient bounds

gradient. For a nodal scalar field v , compute $\nabla v|_T = \sum_{i=0}^2 v_i \nabla \lambda_i$. It is constant on a triangle, so no numerical quadrature is required.

affine_integral. Use $\int_T v = |T|(v_0 + v_1 + v_2)/3$.

add_affine_square. Add the exact integral

$$\int_T v^2 = \frac{|T|}{6} (v_0^2 + v_1^2 + v_2^2 + v_0 v_1 + v_0 v_2 + v_1 v_2). \quad (1)$$

This is the usual consistent P_1 mass matrix. As a quick check, $v_0 = v_1 = v_2 = 1$ gives $|T|$. Applied separately to the two components of an affine flux mismatch, it gives its squared L^2 norm exactly, with only interval arithmetic rounding to enclose.

symmetric_norm. For a real symmetric matrix with entries x, y, z , compute its operator norm from

$$\left\| \begin{pmatrix} x & y \\ y & z \end{pmatrix} \right\|_2 = \frac{|x + z| + \sqrt{(x - z)^2 + 4y^2}}{2}.$$

Taking a maximum over triangles supplies the constants $C_q = \|\mathbb{A}_q\|_{L^\infty}$, C_r , and $C_{qr} = \|\mathbb{A}_{qr}\|_{L^\infty}$ used in Lecture 2.

coefficients. Differentiate the planar pullback coefficient

$$\mathbb{A}(s, t) = \det(\text{Id} + sD\theta_q + tD\theta_r)(\text{Id} + sD\theta_q + tD\theta_r)^{-1}(\text{Id} + sD\theta_q + tD\theta_r)^{-T}.$$

The function returns $\mathbb{A}_q, \mathbb{A}_r, \mathbb{A}_{qr}$ and the mixed determinant derivative

$$(\text{div } \theta_q)(\text{div } \theta_r) - \text{tr}(D\theta_q D\theta_r).$$

The first matrix derivative is $\mathbb{A}_q = (\text{div } \theta_q) \text{Id} - D\theta_q - D\theta_q^T$. To read the second derivative without an extra chain of aliases, expand it as

$$\begin{aligned} \mathbb{A}_{qr} = & ((\text{div } \theta_q)(\text{div } \theta_r) - \text{tr}(D\theta_q D\theta_r)) \text{Id} \\ & - (\text{div } \theta_q)(D\theta_r + D\theta_r^T) - (\text{div } \theta_r)(D\theta_q + D\theta_q^T) \\ & + D\theta_q D\theta_r + D\theta_r D\theta_q + D\theta_q^T D\theta_r^T + D\theta_r^T D\theta_q^T \\ & + D\theta_q D\theta_r^T + D\theta_r D\theta_q^T. \end{aligned}$$

The source stores each symmetric matrix in three entries. Its temporary four-entry arrays named **Q** and **R** are simply the row-major Jacobians $D\theta_q, D\theta_r$; no additional mathematical quantities are needed to understand them.

state_error. Compute

$$\Phi_0 = \| -x/2 + \text{curl } \tilde{\psi}_0 - \nabla \tilde{u} \|_{L^2(P_n)}, \quad \text{curl } \psi = (\partial_y \psi, -\partial_x \psi).$$

The flux has divergence -1 . Continuity of the scalar potential ensures the required normal continuity of its curl. Thus Φ_0 bounds the state energy error even if the auxiliary fit for $\tilde{\psi}_0$ is inexact.

4.4 Residuals, reference values, and the main calculation

interior_residual_norm. Take the Euclidean norm of a residual array on the interior nodes only. The basis of $V_h \subset H_0^1(P_n)$ has no boundary degrees of freedom. Including the boundary rows of the unconstrained stiffness matrix would therefore compute the wrong algebraic residual for this problem.

read_mode_reference. Read the requested component of a `MODE_DATA` record: `rr`, `tt`, `rt_re`, or `rt_im`. These correspond respectively to the two diagonal entries and the real and imaginary parts of the upper-right entry. The entry pass checks the requested index and its conjugate; the final mode pass checks uniqueness and completeness of all Fourier records.

main: assertions before the triangle loop. Require exactly one of `-check-regular-inputs` and `-diagnostic-nonstrict`. Reference certification requires the strict choice, the reference file, the entry name, and the proposed maximum radius. Validate dimensions, metadata, and radii; read the export; reject trailing data. Set the four Dirichlet candidate fields to exact zero on the certified boundary. The scalar flux potentials have no Dirichlet condition. Then validate and reconstruct the exact regular inputs.

main: the triangle loop. Accumulate the area, $\int \tilde{u}$, the second area variation, the raw Hessian centre, the three coefficient norm bounds, the four residual vectors, the node patch areas, and the three remaining squared flux mismatches. The state mismatch was already evaluated by **state_error**. Each calculation uses constant gradients and (1); there is no quadrature remainder to estimate.

The mathematical residual convention is $\rho = \text{right side} - \text{matrix} \times \text{candidate}$. The C arrays `res0`, `resq`, `resr`, `resz` store its *negative*. Norms do not change, but the sign matters when correcting the centre.

main: algebraic errors. The polygon lies in $(-1, 1)^2$, so domain monotonicity gives $\lambda_1(P_n) \geq \pi^2/2$. The local mass matrix gives, for the interior-node stiffness matrix K_h ,

$$K_h \succeq \alpha \text{Id}, \quad \alpha = \frac{\pi^2}{24} \min_{i \text{ interior}} \sum_{T \ni i} |T| > 0.$$

Thus the dual norm of a residual restricted to V_h is bounded by its Euclidean coefficient norm divided by $\sqrt{\alpha}$. The first-derivative algebraic bounds also contain the state error multiplied by C_q or C_r , because the right sides depend on the state candidate.

The corrected centres are those derived in Lecture 2:

$$\tilde{C}_{\text{corr}} = \tilde{C} + \rho_r(\tilde{u}_q) + \rho_0(\tilde{z}), \quad \tilde{J}_{\text{corr}} = \int_P \tilde{u} - \frac{1}{2} \int_P |\nabla \tilde{u}|^2.$$

In the C loop, adding $\rho_r(\tilde{u}_q)$ means subtracting `resr` dotted with `uq`. The remaining algebraic error in the Hessian centre is quadratic in the relevant energy errors.

main: the final entry enclosure. Let $\widehat{\delta}_0, \widehat{\delta}_q, \widehat{\delta}_r$ and $\widehat{\eta}_{qr}$ denote the computed upper bounds for the state, first-derivative, and second-lifting Galerkin errors, and let E_{alg} bound the corrected discrete-centre error. These are the quantities of Lecture 2, not new error definitions. For the nonzero Fourier pairings the first area variations vanish. The program evaluates

$$\begin{aligned}\widetilde{F}_{qr} &= \frac{\widetilde{C}_{\text{corr}}}{A^2} - \frac{2\widetilde{J}_{\text{corr}}A_{qr}}{A^3}, \\ R_F &= \frac{\widehat{\delta}_q\widehat{\delta}_r + \frac{1}{2}C_{qr}\widehat{\delta}_0^2 + \widehat{\delta}_0\widehat{\eta}_{qr} + E_{\text{alg}}}{A^2} + \frac{|A_{qr}|}{A^3}(\widehat{\delta}_0^2 + \epsilon_0^2).\end{aligned}$$

Here $A = |P_n|$, $A_{qr} = D^2A[q, r]$, and ϵ_0 bounds the state algebraic energy error. It enlarges the ball for $\widetilde{F}_{qr}$ by R_F . Its output separates `F_hessian_continuous_error<=` from `F_hessian_algebraic_error<=`, making the source of a large radius visible.

main: the handoff assertion. If a reference is supplied, reconstruct its ball using the proposed common mode radius and prove that it contains the complete entry enclosure. Repeat at the conjugate mode $n - k$; negate the imaginary entry there. Only successful containment prints

```
ENTRY_CERTIFIED k=... conjugate_k=... entry=... max_radius=...
```

The driver checks that all four metadata fields match its request. This second pass closes the gap between a rigorously computed internal ball and the finite decimal data actually sent to the eigenvalue verifier.

5 The Fourier verifier, function by function

Open `MaxTorsionValidation/flint/mode_cert.c`. Its input consists of

```
MODE_DATA k a d Re(b) Im(b)
```

representing $\begin{pmatrix} a & b \\ \bar{b} & d \end{pmatrix}$, and a radius file with one `MODE_RADIUS k radius` line for every index. The letters in this file format mean the two diagonal entries and the complex upper-right entry; in Lecture 1 these are α_k, β_k , and $i\gamma_k$.

usage. Describe the accepted options and exit with status 2. A global radius and a per-mode radius file cannot be used together.

print_ball. Print an enclosure for inspection. The displayed precision does not replace the precision used in the sign predicates.

main: input validation. Read finite real components and nonnegative radii. With the expected number of symbols specified, require exactly one mode and one radius for every $k = 0, \dots, n - 1$. Reject missing, duplicate, or out-of-range indices. These checks prevent a repeated negative mode from standing in for an untested one.

main: analytic symmetry information. With `-exact-regular-similarities`, replace B_0 by the exact zero matrix. At $k = 1, n - 1$, use the exact translation eigenvalue zero and the trace for the other eigenvalue. At the even Nyquist index, set the off-diagonal entries to zero. These are analytic identities from Lecture 1; they are applicable because the first stage proved that the data correspond to the regular polygon and its normalized Fourier directions.

main: eigenvalues and sign counts. For the remaining blocks evaluate

$$\lambda_{\pm} = \frac{a + d \pm \sqrt{(a - d)^2 + 4(\operatorname{Re} b)^2 + 4(\operatorname{Im} b)^2}}{2}.$$

Count an eigenvalue as negative only when its entire Arb ball is strictly negative. The internal word **unresolved** includes the four exact zeros. In a successful regular-polygon run these are its only members. An interval proved positive is not counted as unresolved; it also cannot satisfy the required negative count.

The full driver requires all three counts

$$\text{symbols} = n, \quad \text{negative} = 2n - 4, \quad \text{unresolved} = 4.$$

The program exits 0 when the requested counts hold, 3 when they fail, and 2 on an input error. Its printed **MODE_CERTIFIED** marker means the *requested* counts were satisfied. A standalone invocation that requests weaker counts does not prove local maximality. Nor does this program prove the PDE error radii: those must come from the entry checks.

6 How the rotation-based driver obtains the candidates

The optimization concerns the first derivatives. At vertex 0, the local radial and tangential directions are (1,0) and (0,1). Solve the torsion problem and these two material problems. Rotational symmetry gives the other local vertex derivatives by composing these scalar functions with the inverse rotation. Weighted sums give the Fourier derivatives. Thus only three primal state/first-derivative solves are needed per polygon. The present error estimator still solves a second lifting for each real Hessian pairing, and it computes auxiliary scalar flux fits. Counting those as well would give more than three linear solves.

6.1 The FreeFEM problems

In `freefem/second_variation_lifting_rotations.edp`, the named problem **state** is the torsion Galerkin equation, **firstQ** and **firstR** are the two differentiated Galerkin equations, and **lifting** is the discrete second lifting of Lecture 2. The **-make-basis 1** branch solves the first three once at vertex 0. The **-cached-fields** branch reads the state and the requested first derivatives from the cache and solves only **lifting**.

The named problem **fitCurl** minimizes a regularized flux mismatch over continuous piecewise affine scalar potentials. The added term is $10^{-12} \int_P \psi_h^2 / 2$ in the minimized functional: it makes the fit coercive and fixes the additive constant, while also slightly changing the nonconstant part of the minimizer. Three such fits accompany the initial basis; one more accompanies each second lifting. The argument for equilibrium uses the form of the flux, not successful convergence of this optimization. The solver tolerance affects sharpness and speed, while the C verifier evaluates the actual supplied potential.

The separate `freefem/torsion_hessian_rotations.edp` assembles a floating Hessian from the same three-solve principle. It is useful for inspection and comparisons. Its named problems **torsion**, **materialX**, and **materialY** solve the state and the two coordinate material equations at vertex zero. The driver option **-entry-centers** avoids needing this additional floating Hessian by taking reference centres directly from the entry estimates.

6.2 rotation_cache.py

prepare. Read the basis export, check dimensions and finiteness, identify lattice addresses, construct the node permutation for rotation, and check its cycles and coordinate consistency. Shared-ray addresses have one canonical key. Apply the permutation to the two vertex-zero

scalar solutions and their flux potentials, then form Fourier combinations with NumPy’s FFT. For NumPy’s negative-exponent convention, cosine weights come from the real part and positive sine weights from the negative imaginary part. The normalization is $\sqrt{n}/2$, or $\sqrt{n}$ for the even Nyquist mode, when dividing an unnormalized Fourier sum.

The nested function `direction` selects the radial/tangential and cosine/sine combination for one requested field. The cache is only a candidate-generation device: exact geometric and directional checks, and residual bounds for the resulting fields, are repeated in C.

6.3 `certificate_build.py`

build_verifiers. Compile `second_lifting_cert` and `mode_cert` from the source copies saved inside the new archive, using a forced build. Return their private executable paths to the driver and record their hashes. A small diagnostic records the compiler and FLINT header/runtime versions. Failure of that diagnostic is recorded as unavailable metadata; the two verifier builds themselves must succeed. The run therefore does not silently use a pre-existing executable from a different source version.

6.4 `certify_with_rotations.py`

run. Execute a subprocess, save its output in a designated log, measure elapsed time, and raise an error for a nonzero exit status. The final mode step is handled separately so that an inconclusive certificate can be recorded normally rather than mistaken for a crashed program.

ball. Read the printed midpoint and radius for a named output using decimal arithmetic. Its result is used to propose a radius. It has no independent authority to prove containment.

main: setup and candidate phase. Parse the range of n , m , worker count, CPU affinity limit, flux-fit tolerance, and a new archive path. Limit numerical-library threads to one per process, set the chosen affinity, and run with reduced scheduling priority. Save a source snapshot and manifest, build the private verifiers, and create the three-solve rotation cache for each polygon.

The nested function `evaluate` generates one second-lifting export, checks its header against the requested pairing, and runs the entry verifier in strict mode. From the displayed centre and error ball it proposes a slightly enlarged decimal radius covering both k and $n - k$. Four pairings are used for each representative nonzero mode, except the even Nyquist mode, which needs two.

main: reference and verification phase. Once all centres exist, write `modes.txt` once and retain it unchanged during verification. Take the maximum proposed entry radius for each mode and its conjugate. The nested function `verify` reruns the C entry verifier with this exact reference file and radius, checks the complete `ENTRY_CERTIFIED` marker, and compresses the checked export. After all entries pass, run `mode_cert` at 192 bits with the required $n, 2n - 4, 4$ counts. Save `RESULT.json`, append `results.jsonl`, and stop on an inconclusive polygon if `-stop-on-failure` is set. Finally save archive hashes and a `FINISHED` marker.

Reading a run’s status. The Python driver can exit normally after recording `NOT_CERTIFIED`; this is an expected numerical outcome. `FINISHED` means that the requested sweep or its stopping rule has finished. To claim success for a particular n , inspect its `RESULT.json`: it must say `status: CERTIFIED` and `mode_exit_code: 0`, with the corresponding entry logs and final counts present. A shell exit code alone is insufficient.

7 Other validation utilities and their precise roles

The two C programs above are the maintained complete certification path. The repository also contains smaller verifiers and diagnostic utilities. Their output should be read according to the narrower assertion each one actually establishes.

7.1 `residual_cert.c`

`read_arb` parses a scalar. Its `main` reads a dense matrix, right side, candidate vector, and a supplied positive lower bound α ; it evaluates the residual and the resulting algebraic error bound. The assumption that the matrix is symmetric positive definite and that $\alpha \leq \lambda_{\min}$ is supplied by the caller. Positivity of the number α alone does not prove that assumption. This utility does not perform the regular-fan and boundary checks of the main verifier.

7.2 `second_variation_cert.c`

`die` reports invalid input; `set_nonnegative` parses and checks nonnegative bounds. The `main` function reads complete records of two direction indices and five bounds, and evaluates

$$\delta_q \delta_r + \frac{1}{2} C_{qr} \delta_0^2 + \delta_0 \eta_{qr}.$$

It checks the arithmetic combination, assuming that the five supplied bounds have already been proved. It does not establish their PDE meaning.

7.3 `majorant_cert.c`

Function	Role
<code>die</code>	Reject an invalid export.
<code>read_arb</code> , <code>read_field</code>	Read scalar balls and arrays.
<code>grad_field</code>	Compute a P_1 gradient on a triangle.
<code>add_affine_square</code>	Integrate a squared affine mismatch.
<code>energy_norm</code>	Integrate a field's squared gradient and take its norm.
<code>state_majorant</code>	Bound the state error by its equilibrated flux mismatch.
<code>material_majorant</code>	Bound a first material error, including propagation of the state error.
<code>print_ball</code>	Display an enclosure.
<code>main</code>	Read candidate data and combine direct state/material majorants into diagnostic bounds.

This program is useful for understanding individual majorants and direct Gram estimates. It does not establish the complete canonical-fan input contract used by `second_lifting_cert`; its output alone is not the maintained local-maximality certificate.

7.4 Drivers, summaries, and checks

`freefem/test_extended_certificates.py` follows the same two-stage certificate structure with per-entry candidate solves instead of a shared rotation cache. Its `run`, `ball`, nested `evaluate` and `verify`, and `main` have the corresponding roles described above. The original candidate-generating FreeFEM files remain available for comparisons.

In `freefem/summarize_extended_certificates.py`, `audit` reads an archive's metadata, source hashes, entry markers, radii, and mode output; `main` assembles the requested result

tables. Table upper endpoints are rounded outward using decimal arithmetic. This is an audit of records, not a fresh replay of every finite element export.

The input-rejection tests in `flint/test_certificate_inputs.py` and the rotation comparisons in `freefem/test_torsion_hessian_rotations.py` exercise implementation properties. Neither a successful comparison with floating output nor a successful test suite substitutes for the entry enclosures and sign counts for a particular polygon.

8 Running a certificate yourself

The commands below are run from the repository root. They require FreeFEM, a C compiler with FLINT/Arb development files, Python, and NumPy; the maintained installation instructions are in `MaxTorsionValidation/README.md`. The archive directory must not already exist, so each run preserves its own evidence.

For a small first run:

```
python3 MaxTorsionValidation/freefem/certify_with_rotations.py \
  5 5 --m 32 --entry-centers --jobs 6 --cpu-cores 6 \
  --stop-on-failure --archive /tmp/torsion_n5_m32_course
```

For the largest successfully archived case:

```
python3 MaxTorsionValidation/freefem/certify_with_rotations.py \
  25 25 --m 128 --entry-centers --jobs 6 --cpu-cores 6 \
  --flux-eps 1e-13 --stop-on-failure \
  --archive /tmp/torsion_n25_m128_course
```

To test a range, replace the two positional integers by its first and last n . The stopping option halts after the first failure to certify. For this project's six-physical-core machine the shown settings use six workers. More generally, `-cpu-cores` selects from OS CPU affinity indices; it does not discover physical-core topology on another machine.

Inspect a completed run with, for example,

```
cat /tmp/torsion_n25_m128_course/n25/RESULT.json
tail -n 2 /tmp/torsion_n25_m128_course/n25/mode_cert.log
```

The expected final counts for $n = 25$ are 25 symbols, 46 negative eigenvalues, and four exact similarity zeros. The archive also contains the reference centres, per-mode radii, compressed entry exports, entry logs, source snapshot, build metadata, and hashes.

An independent replay of a single entry uses the archived verifier and the decompressed export, with the same `-prec 192`, geometry and direction radii, strict input option, reference file, entry name, and `-max-radius` as its original command. Running only the final `mode_cert` is a useful replay of the small spectral calculation, but it does not replay the entry PDE bounds.

9 What the archived computations establish

The following tables transcribe the manuscript and the archived table data. No new certification sweep was run to write these notes. All eigenvalue values refer to $D^2(J/A^2)$ in the normalized vertex coordinates of Lecture 1. The listed upper bound is the largest certified upper endpoint among eigenvalues other than the four similarity zeros. Negative upper bounds are rounded toward $+\infty$, so the displayed values remain safe upper bounds.

n	m	Negative / required	Upper bound
3	32	2/2	$-1.00 \cdot 10^{-2}$
4	32	4/4	$-6.20 \cdot 10^{-3}$
5	32	6/6	$-3.10 \cdot 10^{-3}$
6	32	8/8	$-1.70 \cdot 10^{-3}$
7	32	10/10	$-9.80 \cdot 10^{-4}$
8	32	12/12	$-5.90 \cdot 10^{-4}$
9	32	14/14	$-3.60 \cdot 10^{-4}$
10	32	16/16	$-2.20 \cdot 10^{-4}$

The $n = 3, 4$ cases are also known analytically. The computations extend the local-maximality result through $n = 25$. Four similarity zeros accompany every successful row; they are known exactly from invariance, not inferred from small decimal numbers.

The $m = 64$ computations

n	Triangles	Negative / required	Upper bound	Outcome
11	45056	18 / 18	$-2.11 \cdot 10^{-4}$	certified
12	49152	20 / 20	$-1.50 \cdot 10^{-4}$	certified
13	53248	22 / 22	$-1.08 \cdot 10^{-4}$	certified
14	57344	24 / 24	$-7.99 \cdot 10^{-5}$	certified
15	61440	26 / 26	$-5.97 \cdot 10^{-5}$	certified
16	65536	28 / 28	$-4.51 \cdot 10^{-5}$	certified
17	69632	30 / 30	$-3.44 \cdot 10^{-5}$	certified
18	73728	32 / 32	$-2.64 \cdot 10^{-5}$	certified
19	77824	30 / 34	$1.35 \cdot 10^{-5}$	inconclusive
20	81920	28 / 36	$1.69 \cdot 10^{-4}$	inconclusive

The $m = 128$ computations

n	Triangles	Negative / required	Upper bound	Minutes	Outcome
19	311296	34 / 34	$-2.73 \cdot 10^{-5}$	15.7	certified
20	327680	36 / 36	$-2.22 \cdot 10^{-5}$	18.0	certified
21	344064	38 / 38	$-1.81 \cdot 10^{-5}$	20.8	certified
22	360448	40 / 40	$-1.49 \cdot 10^{-5}$	13.5	certified
23	376832	42 / 42	$-1.24 \cdot 10^{-5}$	15.8	certified
24	393216	44 / 44	$-1.03 \cdot 10^{-5}$	19.4	certified
25	409600	46 / 46	$-8.71 \cdot 10^{-6}$	17.7	certified

At $m = 64$, the archived checks succeed through $n = 18$ and are inconclusive for $n = 19, 20$. A positive upper endpoint in those rows does not prove a positive eigenvalue. It means the enclosure is too wide to establish all the desired negative signs. Refinement to $m = 128$ succeeds for every tested $n = 19, \dots, 25$. These computations neither establish a maximal possible n at either mesh size nor give a conclusion beyond $n = 25$.

9.1 Time and the meaning of the three-solve optimization

The recorded $m = 128$ run for $n = 25$ took 1059.4 seconds, about 17.7 minutes, on the six-physical-core Intel Core i7-9750H machine (2.60 GHz). It has 409600 triangles, 206401 nodes, and 48 real entry checks. The full optimized $n = 22, \dots, 25$ records total about 66.4 minutes; allowing roughly 70 minutes is a useful estimate on that machine. The $n = 24$ record includes a handoff interval, and the earlier $n = 19, 20, 21$ runs used a different allocation/workflow, so the timings do not form a controlled scaling benchmark.

The three initial primal solves do not make the whole proof constant cost in n : there are more modes, more mesh triangles, one second lifting and one auxiliary flux fit per real pairing, and two Arb entry passes. This explains why the optimization is substantial while certification still takes minutes.

9.2 Where the evidence is stored

The main archive groups are under `MaxTorsionValidation/results/`:

- `review_2026_09_08_certificate_archive`: the $m = 32$ chain.
- `extended_m128_fast_n19_onward`: the $m = 128$, $n = 19, 20, 21$ records.
- `extended_m128_rotations_n22_n30`: the completed $n = 22, 23, 24$ records; the folder name records an initial range, not results through 30.
- `extended_m128_rotations_n25`: the separate $n = 25$ certificate.
- `extended_m64_table_data.json` and `extended_m128_table_data.json`: the audited table rows and their per-row log references.

The packaging rename from `code/` to `MaxTorsionValidation/` preserved historical archives byte for byte. Consequently an old JSON record can still contain a `code/results/...` path; resolve that prefix to the current source root when locating the file. The packaging provenance explains this explicitly.

The recorded validity review checked 672 entry-log records, replayed 25 final mode checks, and performed four complete entry replays. Its archive hash audit covered 2701 files and recorded one historical checksum mismatch for the compressed $n = 21$, $k = 6$, radial-radial export. That entry was freshly recertified at the archived radius; the original checksum record was retained and the replay documented in `validity_review_20260909/n21-replay-provenance.json`. This is the precise scope of that review: it was not a fresh recomputation of all 672 entry exports. Hashes establish correspondence of bytes; the mathematical conclusion comes from the verification chain applied to the identified data.

Exercises and audit questions

Exercise 9.1. *An FFT bug changes a cached first derivative. Why can the full chain still be valid if it reports success?*

Answer. The cached field is merely a conforming candidate. The C verifier computes its residual and flux mismatch for the prescribed exact direction. A poor candidate normally widens the enclosure or causes failure; it is not assumed to solve the intended PDE.

Exercise 9.2. *The entry pass gives an enclosure centred at c with radius r . The mode file stores a decimal c_0 . Is using radius r around c_0 sufficient?*

Answer. In general no: the difference between c_0 and the entry centre must be covered as well. The second C pass proves containment of the whole internal enclosure in the ball built from the actual mode file, and also checks the conjugate entry.

Exercise 9.3. *Why are four exact zeros compatible with strict local maximality?*

Answer. They are precisely the tangent directions to similarities, along which J/A^2 is constant. On a local slice fixing translation, rotation, and scale, the Hessian is strictly negative definite.

Exercise 9.4. *What follows if `RESULT.json` says `NOT_CERTIFIED` at $m = 64$?*

Answer. This chain at this mesh and these candidates has not proved the required signs. It supplies no counterexample to local maximality. The successful $m = 128$ checks for $n = 19, 20$ illustrate the difference between a failed enclosure test and a false mathematical claim.

References

- [1] The FLINT development team. Arb: arbitrary-precision real numbers, FLINT documentation. <https://flintlib.org/doc/arb.html>, 2026. Accessed 13 September 2026. Recorded certification runs identify their own FLINT version.